Universidad del Valle de Guatemala

Daniel Eduardo Sacol Cojón

Laboratorio 4 – Polimorfismo

Análisis

Requisitos Funcionales

1. **Registrar equipos.** Almacenar el código, nombre y empresa de cada equipo, e inicializar sus puntos de tabla en cero. Rechazar el registro si el código está vacío o ya pertenece a otro equipo.
2. **Crear encuentros.** Permitir seleccionar fútbol sala o baloncesto y dos equipos previamente registrados y distintos. Cada encuentro comenzará en estado «sin programar» y conservará sus participantes, deporte y estado, además de la programación y el marcador cuando se asignen.
3. **Crear exhibiciones.** Registrar exhibiciones con un título y estado inicial «sin programar». Conservar su programación cuando se asigne. Las exhibiciones no tendrán equipos, marcador ni resultado, y no otorgarán puntos de tabla.
4. **Programar eventos.** Asignar una fecha válida, una hora entera entre 0 y 23 y una sede a un encuentro o exhibición. Si se cumplen las condiciones de programación, guardar los datos y cambiar el estado a «programado» o «programada».
5. **Validar disponibilidad.** Rechazar una programación si otro evento ocupa la misma sede en la misma fecha y hora. Para los encuentros, rechazarla también si cualquiera de los equipos participantes tiene otro encuentro en ese horario.
6. **Reprogramar eventos.** Permitir modificar la fecha, hora o sede de las exhibiciones programadas y de los encuentros programados que no tengan resultado. Validar nuevamente la disponibilidad, excluyendo al propio evento de la comprobación. Rechazar la reprogramación de encuentros finalizados y conservar la programación anterior si la solicitud es inválida.
7. **Registrar resultados y asignar puntos.** Aceptar un marcador de dos enteros no negativos únicamente en encuentros programados. En fútbol sala, asignar 3 puntos al ganador y 0 al perdedor, o 1 a cada equipo si empatan. En baloncesto, rechazar empates y asignar 2 puntos al ganador y 1 al perdedor. Tras validar el resultado, guardar el marcador, sumar los puntos y cambiar el estado a «finalizado». Rechazar resultados para encuentros sin programar o finalizados.
8. **Mostrar la tabla de posiciones.** Presentar todos los equipos con sus puntos de tabla acumulados en ambos deportes. Ordenarlos por puntos de mayor a menor y, en caso de empate, por nombre en orden alfabético.
9. **Mostrar el calendario.** Presentar conjuntamente los encuentros y las exhibiciones con programación asignada, incluidos los encuentros finalizados. Ordenarlos por fecha ascendente, luego por hora ascendente y finalmente por nombre de sede en orden alfabético. Excluir los eventos sin programar.
10. **Ofrecer un menú de consola.** Permitir registrar equipos, crear encuentros y exhibiciones, programar y reprogramar eventos, registrar resultados y consultar la tabla de posiciones y el calendario. Mantener el menú disponible hasta que el usuario seleccione salir.

Reglas y comportamiento

Reglas que dependen del deporte

En el caso de un encuentro de futbol sala se podrá registrar victorias, derrotas y empates. Asignará 3 puntos de tabla al ganador y 0 al perdedor; en caso de empate, asignará 1 punto a cada equipo. Para encuentros de baloncesto no se podrá registrar empates y se asignará 2 puntos para el ganador y 1 al perdedor.

Estas reglas se resolverán en las clases concretas EncuentroFutbolSala y EncuentroBaloncesto, que heredarán de la clase abstracta Encuentro. Cada subclase implementará los métodos de validación del marcador y cálculo de puntos definidos como abstractos en Encuentro, porque su comportamiento cambia según el deporte.

Comportamiento compartido entre encuentros y exhibiciones

Los encuentros y las exhibiciones pueden programarse y reprogramarse, conservar una fecha, hora y sede, y aparecer en el calendario. Este comportamiento se expresará mediante una interfaz llamada Programable, implementada por Encuentro y por la clase concreta Exhibicion. La clase Exhibicion conservará su título, estado y programación. No heredará de Encuentro, porque no tiene equipos, marcador ni resultado.

Comportamiento exclusivo de los encuentros respecto a los resultados

Una segunda interfaz, llamada Puntuable, establecerá las operaciones para recibir resultados y repartir puntos de tabla. La clase abstracta Encuentro implementará esta interfaz y utilizará las reglas definidas por cada subclase concreta. La clase Exhibicion no implementará Puntuable, ya que las exhibiciones únicamente aparecen en el calendario y no intervienen en la tabla de posiciones.

Estados de un encuentro

Un encuentro puede encontrarse en tres estados: sin programar, programado y finalizado.

Estado inicial	**Operación y condiciones**	**Estado resultante**
No existe	Crear un encuentro con un deporte y dos equipos registrados distintos.	Sin programar
Sin programar	Asignar una fecha, una hora válida y una sede, sin conflictos de sede ni de equipos.	Programado
Programado	Reprogramar el encuentro con una nueva programación válida, mientras no tenga resultado.	Programado
Programado	Registrar un marcador válido según las reglas del deporte y asignar los puntos de tabla correspondientes.	Finalizado

Estados de una exhibición

Una exhibición puede encontrarse en dos estados: sin programar y programada.

Estado inicial	**Operación y condiciones**	**Estado resultante**
No existe	Crear una exhibición indicando su título.	Sin programar
Sin programar	Asignar una fecha, una hora válida y una sede disponible en ese horario.	Programada
Programada	Reprogramar la exhibición con una fecha, hora y sede válidas, sin conflictos de sede.	Programada

Las exhibiciones no tienen un estado «finalizada», porque el sistema no registra resultados para ellas. El paso de la fecha programada no provoca un cambio automático de estado.

En ambos tipos de evento, la reprogramación modifica los datos de programación, pero mantiene el estado. Si una operación es inválida, el sistema muestra un mensaje de error y conserva tanto el estado como los datos anteriores.

Atributos y métodos

Tabla de atributos

Clase o interfaz	Atributo	Tipo	Visibilidad y modificadores	Propósito
Programable	—	—	—	No declara atributos.
Puntuable	—	—	—	No declara atributos.
Equipo	codigo	String	private final	Identificar al equipo de manera única dentro de la copa.
Equipo	nombre	String	private final	Guardar el nombre del equipo.
Equipo	empresa	String	private final	Guardar la empresa representada.
Equipo	puntosTabla	int	private	Acumular los puntos obtenidos en los encuentros.
Encuentro	equipo1	Equipo	private final	Referenciar al primer participante.
Encuentro	equipo2	Equipo	private final	Referenciar al segundo participante.
Encuentro	marcador1	Integer	private	Guardar la anotación del primer equipo; será null si no existe resultado.
Encuentro	marcador2	Integer	private	Guardar la anotación del segundo equipo; será null si no existe resultado.
Encuentro	estado	EstadoEvento	private	Indicar si el encuentro está sin programar, programado o finalizado.
Encuentro	fecha	LocalDate	private	Guardar la fecha asignada.
Encuentro	hora	int	private	Guardar la hora de inicio.
Encuentro	sede	String	private	Guardar la sede asignada.
EncuentroFutbolSala	—	—	—	No declara atributos adicionales; los datos comunes se administran desde Encuentro.
EncuentroBaloncesto	—	—	—	No declara atributos adicionales; los datos comunes se administran desde Encuentro.
Exhibicion	titulo	String	private final	Identificar la actividad.
Exhibicion	estado	EstadoEvento	private	Indicar si está sin programar o programada.

Exhibicion	fecha	LocalDate	private	Guardar la fecha asignada.
Exhibicion	hora	int	private	Guardar la hora de inicio.
Exhibicion	sede	String	private	Guardar la sede asignada.
EstadoEvento	SIN_PROGRAMAR, PROGRAMADO, FINALIZADO	EstadoEvento	Constantes de enumeración	Representar los estados permitidos. Las exhibiciones solo utilizan los dos primeros.
Copa	equipos	Map<String, Equipo>	private final	Guardar los equipos utilizando el código como clave.
Copa	encuentros	List<Encuentro>	private final	Guardar los encuentros de los diferentes deportes.
Copa	exhibiciones	List<Exhibicion>	private final	Guardar las exhibiciones.
VistaConsola	entrada	Scanner	private final	Leer la información ingresada por el usuario.
ControladorCopa	copa	Copa	private final	Referenciar el modelo administrado.
ControladorCopa	vista	VistaConsola	private final	Referenciar la vista utilizada para interactuar con el usuario.
Main	—	—	—	No necesita atributos; utiliza variables locales en main.

Tabla de métodos y constructores

Clase o interfaz	Método o constructor	Retorno	Visibilidad y modificadores	Propósito
Programable	programar(LocalDate fecha, int hora, String sede)	void	public abstract	Definir la operación para asignar o modificar la programación.
Programable	getFecha()	LocalDate	public abstract	Consultar la fecha asignada.
Programable	getHora()	int	public abstract	Consultar la hora de inicio.
Programable	getSede()	String	public abstract	Consultar la sede.
Programable	getDescripcion()	String	public abstract	Obtener un texto que identifique el evento.
Programable	getEquiposParticipantes()	List<Equipo>	public abstract	Consultar los equipos involucrados para validar conflictos.
Programable	compareTo(Programable otro)	int	public default	Establecer un único orden por fecha, hora y sede para

				eventos con programación.
Puntuable	registrarResultado(int marcador1, int marcador2)	void	public abstract	Definir el registro de resultados y la asignación de puntos.
Equipo	Equipo(String codigo, String nombre, String empresa)	—	public	Inicializar los datos y establecer los puntos de tabla en cero.
Equipo	getCodigo()	String	public	Consultar el código.
Equipo	getNombre()	String	public	Consultar el nombre.
Equipo	getEmpresa()	String	public	Consultar la empresa representada.
Equipo	getPuntosTabla()	int	public	Consultar los puntos acumulados.
Equipo	sumarPuntos(int puntos)	void	De paquete	Agregar puntos no negativos como consecuencia de un resultado válido.
Equipo	compareTo(Equipo otro)	int	public	Ordenar por puntos descendentes y, en caso de empate, por nombre alfabético.
Encuentro	Encuentro(Equipo equipo1, Equipo equipo2)	—	protected	Validar los participantes e inicializar el encuentro sin programación ni resultado.
Encuentro	programar(LocalDate fecha, int hora, String sede)	void	public	Aplicar una programación válida si el encuentro no está finalizado.
Encuentro	registrarResultado(int marcador1, int marcador2)	void	public final	Validar el resultado, calcular y asignar puntos, guardar el marcador y finalizar el encuentro.
Encuentro	validarReglasDeporte(int marcador1, int marcador2)	void	protected abstract	Delegar en cada subclase las restricciones particulares de su deporte.
Encuentro	calcularPuntos(int marcador1, int marcador2)	int[]	protected abstract	Devolver los puntos del primer y segundo equipo, respectivamente.
Encuentro	getEstado()	EstadoEvento	public	Consultar el estado actual.
Encuentro	getMarcador1()	Integer	public	Consultar la anotación del primer equipo.
Encuentro	getMarcador2()	Integer	public	Consultar la anotación del segundo equipo.

Encuentro	getFecha()	LocalDate	public	Consultar la fecha asignada.
Encuentro	getHora()	int	public	Consultar la hora de inicio.
Encuentro	getSede()	String	public	Consultar la sede asignada.
Encuentro	getEquiposParticipantes()	List<Equipo>	public	Devolver una lista no modificable con los dos participantes.
Encuentro	getDescripcion()	String	public abstract	Delegar en las subclases la descripción del encuentro y su deporte.
EncuentroFutbolSala	EncuentroFutbolSala(Equipo equipo1, Equipo equipo2)	—	public	Invocar super(equipo1, equipo2) para inicializar los datos comunes.
EncuentroFutbolSala	validarReglasDeporte(int marcador1, int marcador2)	void	protected	Aceptar los marcadores que cumplen la validación común, incluidos los empates.
EncuentroFutbolSala	calcularPuntos(int marcador1, int marcador2)	int[]	protected	Asignar 3 puntos por victoria, 1 por empate y 0 por derrota.
EncuentroFutbolSala	getDescripcion()	String	public	Identificar el deporte y los equipos participantes.
EncuentroBaloncesto	EncuentroBaloncesto(Equipo equipo1, Equipo equipo2)	—	public	Invocar super(equipo1, equipo2) para inicializar los datos comunes.
EncuentroBaloncesto	validarReglasDeporte(int marcador1, int marcador2)	void	protected	Rechazar los marcadores empatados.
EncuentroBaloncesto	calcularPuntos(int marcador1, int marcador2)	int[]	protected	Asignar 2 puntos al ganador y 1 al perdedor.
EncuentroBaloncesto	getDescripcion()	String	public	Identificar el deporte y los equipos participantes.
Exhibicion	Exhibicion(String titulo)	—	public	Guardar el título e inicializar la exhibición sin programar.
Exhibicion	programar(LocalDate fecha, int hora, String sede)	void	public	Asignar o modificar una programación válida.
Exhibicion	getEstado()	EstadoEvento	public	Consultar el estado actual.
Exhibicion	getFecha()	LocalDate	public	Consultar la fecha asignada.

Exhibicion	getHora()	int	public	Consultar la hora de inicio.
Exhibicion	getSede()	String	public	Consultar la sede asignada.
Exhibicion	getDescripcion()	String	public	Devolver el título de la exhibición.
Exhibicion	getEquiposParticipantes()	List<Equipo>	public	Devolver una lista vacía, porque la exhibición no tiene equipos.
EstadoEvento	—	—	—	No declara métodos propios adicionales.
Copa	Copa()	—	public	Inicializar las colecciones vacías.
Copa	registrarEquipo(String codigo, String nombre, String empresa)	Equipo	public	Validar que el código no esté vacío ni repetido y registrar el equipo.
Copa	buscarEquipo(String codigo)	Equipo	public	Localizar un equipo registrado o informar su ausencia mediante un error.
Copa	agregarEncuentro(Encuentro encuentro)	void	public	Registrar un encuentro cuyos participantes pertenezcan a la copa.
Copa	agregarExhibicion(Exhibicion exhibicion)	void	public	Registrar una exhibición.
Copa	getEncuentros()	List<Encuentro>	public	Devolver una copia no modificable para seleccionar encuentros.
Copa	getEventos()	List<Programable>	public	Devolver una lista no modificable con todos los encuentros y exhibiciones.
Copa	programarEvento(Programable evento, LocalDate fecha, int hora, String sede)	void	public	Validar la programación y solicitar al evento que la aplique.
Copa	validarProgramacion(Programable evento, LocalDate fecha, int hora, String sede)	void	private	Comprobar los datos, la pertenencia del evento y los conflictos de sede y equipos, excluyendo al propio evento.
Copa	getTablaPosiciones()	List<Equipo>	public	Devolver una copia de los equipos ordenada mediante su criterio de comparación.
Copa	iterator()	Iterator<Programable>	public	Recorrer una copia no modificable de los eventos con fecha asignada,

				ordenados cronológicamente.
VistaConsola	VistaConsola(Scanner entrada)	—	public	Recibir el lector de entradas.
VistaConsola	mostrarMenu()	void	public	Presentar las opciones disponibles.
VistaConsola	leerTexto(String mensaje)	String	public	Solicitar y devolver un texto.
VistaConsola	leerEntero(String mensaje)	int	public	Solicitar un entero y repetir la lectura ante un formato incorrecto.
VistaConsola	leerFecha(String mensaje)	LocalDate	public	Solicitar una fecha y controlar errores de formato.
VistaConsola	mostrarMensaje(String mensaje)	void	public	Mostrar confirmaciones o errores.
VistaConsola	mostrarEncuentros(List<Encuentro> encuentros)	void	public	Mostrar encuentros numerados para su selección.
VistaConsola	mostrarEventos(List<Programable> eventos)	void	public	Mostrar eventos numerados para programarlos o reprogramarlos.
VistaConsola	mostrarTabla(List<Equipo> equiposOrdenados)	void	public	Presentar la tabla respetando el orden recibido.
VistaConsola	mostrarCalendario(Iterable<Programable> calendario)	void	public	Presentar los eventos respetando el orden del recorrido.
ControladorCopa	ControladorCopa(Copa copa, VistaConsola vista)	—	public	Recibir y conservar las referencias al modelo y la vista.
ControladorCopa	iniciar()	void	public	Cargar los datos iniciales y mantener el menú activo hasta salir.
ControladorCopa	cargarDatosIniciales()	void	private	Crear los datos de ejemplo mediante las operaciones del modelo, sin pedir entradas.
ControladorCopa	ejecutarOpcion(int opcion)	void	private	Coordinar la operación seleccionada en el menú.
ControladorCopa	registrarEquipo()	void	private	Solicitar los datos a la vista y registrar el equipo mediante el modelo.
ControladorCopa	crearEncuentro()	void	private	Solicitar el deporte y los equipos, construir la subclase correspondiente y registrarla.

ControladorCopa	crearExhibicion()	void	private	Solicitar el título, construir la exhibición y registrarla.
ControladorCopa	programarEvento()	void	private	Solicitar el evento y su programación, y llamar a Copa.programarEvento.
ControladorCopa	registrarResultado()	void	private	Seleccionar un encuentro, solicitar el marcador e invocar el contrato Puntuable.
ControladorCopa	mostrarTabla()	void	private	Obtener la tabla del modelo y entregarla a la vista.
ControladorCopa	mostrarCalendario()	void	private	Entregar la copa como Iterable<Programable> a la vista.
Main	Main()	—	private	Impedir la creación de instancias de la clase de inicio.
Main	main(String[] args)	void	public static	Crear y conectar el modelo, la vista y el controlador; iniciar la aplicación y cerrar el lector al terminar.

Justificación de uso de 2 interfaces

Se utilizan dos interfaces porque programar eventos y registrar resultados son responsabilidades diferentes.

Programable permite programar y mostrar encuentros y exhibiciones en el calendario. Puntuable permite registrar resultados y asignar puntos de tabla, por lo que únicamente la implementan los encuentros.

Si ambas responsabilidades se unieran, Exhibicion tendría que implementar operaciones como registrarResultado(), aunque no tenga equipos ni marcador. Esto obligaría a dejar el método sin efecto o a generar un error.

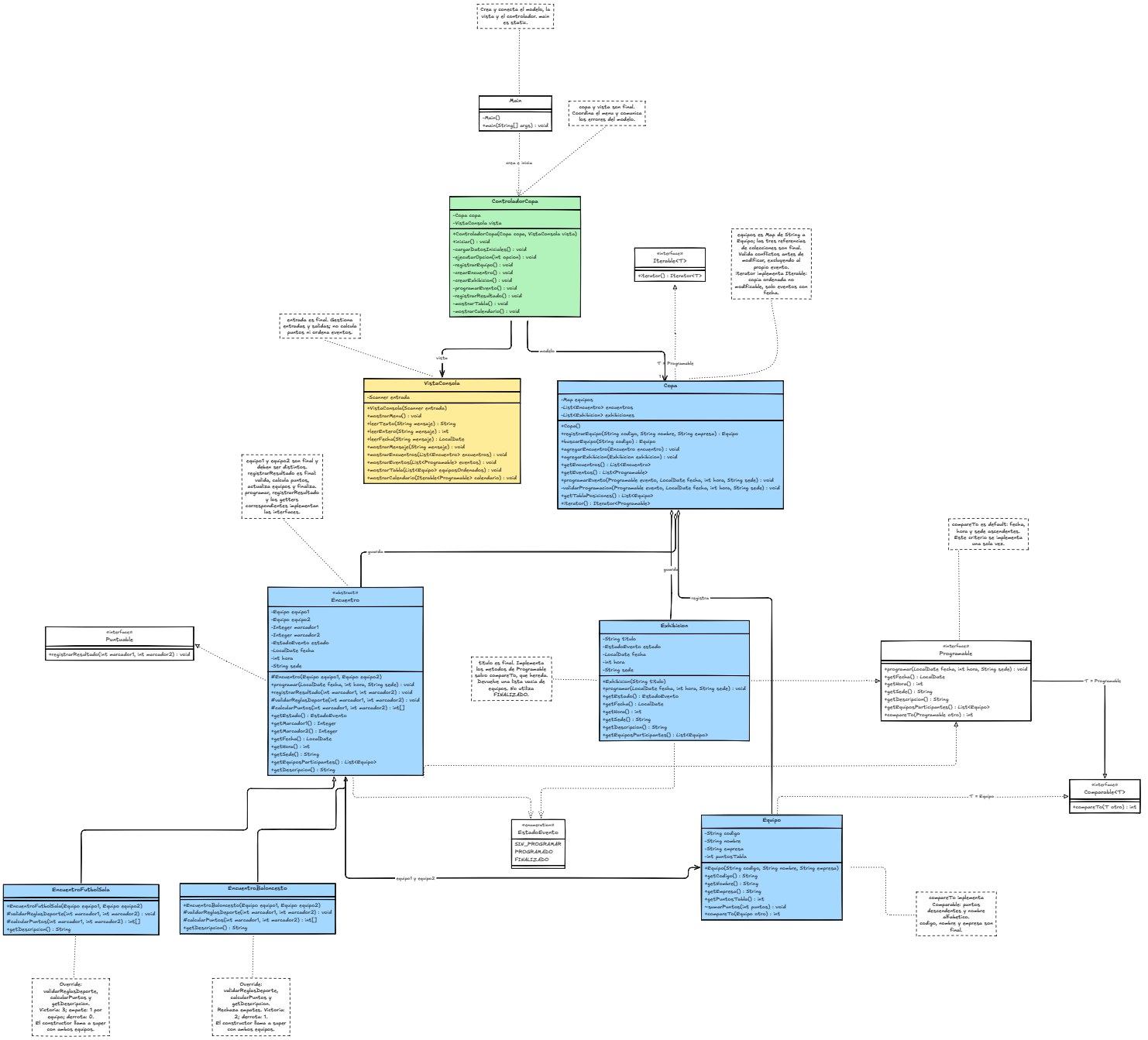
Separar las interfaces permite trabajar con las capacidades de cada objeto sin comprobar su clase concreta y evita exigirle operaciones que no le corresponden.

Modificaciones al agregar otro deporte

Para agregar otro deporte, se crearía una clase concreta que herede de Encuentro e implemente los métodos abstractos para validar el marcador, calcular los puntos y describir el encuentro. También se actualizarían las opciones de selección y el método crearEncuentro() de ControladorCopa para construir la nueva clase.

La tabla de posiciones no necesitaría cambios, porque ordena a los equipos por sus puntos acumulados, independientemente del deporte que los haya generado.

El calendario tampoco necesitaría cambios, porque trabaja con el contrato Programable y mantiene el mismo orden por fecha, hora y sede. Las validaciones de conflictos de sede y equipos seguirían aplicándose al nuevo deporte.


Crea y conecta el modelo, la vista y el controlador; main es static.
Main
-Main()
+main(String[] args) : void
copa y vista son final. Coordina el menú y comunica los errores del modelo.
crea e inicia
ControladorCopa
-Copa copa
-VistaConsola vista
+ControladorCopa(Copa copa, VistaConsola vista)
+iniciar() : void
-cargarDatosIniciales() : void
-ejecutarOpcion(int opcion) : void
-registrarEquipo() : void
-crearEncuentro() : void
-crearExhibicion() : void
-programarEvento() : void
-registrarResultado() : void
-mostrarTabla() : void
-mostrarCalendario() : void
«interface»
IterableCT>
+Iterator() : IteratorCT>
equipos es Map de String a Equipo; las tres referencias de colecciones son final. Valida conflictos antes de modificar, excluyendo al propio evento. Iterator implementa Iterable: copia ordenada no modificable, solo eventos con fecha.
entrada es final. gestiona entradas y salidas; no calcula puntos ni ordena eventos.
vista
modelo
T = Programable
VistaConsola
-Scanner entrada
+VistaConsola(Scanner entrada)
+mostrarMenu() : void
+leerTexto(String mensaje) : String
+leerEntero(String mensaje) : int
+leerFecha(String mensaje) : LocalDate
+mostrarMensaje(String mensaje) : void
+mostrarEncuentros(List<Encuentro> encuentros) : void
+mostrarEventos(List<Programable> eventos) : void
+mostrarTabla(List<Equipo> equiposOrdenados) : void
+mostrarCalendario(Iterable<Programable> calendario) : void
Copa
-Map equipos
-List<Encuentro> encuentros
-List<Exhibicion> exhibiciones
+Copa()
+registrarEquipo(String codigo, String nombre, String empresa) : Equipo
+buscarEquipo(String codigo) : Equipo
+agregarEncuentro(Encuentro encuentro) : void
+agregarExhibicion(Exhibicion exhibicion) : void
+getEncuentros() : List<Encuentro>
+getEventos() : List<Programable>
+programarEvento(Programable evento, LocalDate fecha, int hora, String sede) : void
-validarProgramacion(Programable evento, LocalDate fecha, int hora, String sede) : void
+getTablaPosiciones() : List<Equipo>
+Iterator() : IteratorCProgramable>
equipo1 y equipo2 son final y deben ser distintos. registrarResultado es final: valida, calcula puntos, actualiza equipos y finaliza. programar, registrarResultado y los getters correspondientes implementan las interfaces.
guarda
guarda
registra
«abstract»
Encuentro
-Equipo equipo1
-Equipo equipo2
-Integer marcador1
-Integer marcador2
-EstadoEvento estado
-LocalDate fecha
-int hora
-String sede
#Encuentro(Equipo equipo1, Equipo equipo2)
+programar(LocalDate fecha, int hora, String sede) : void
+registrarResultado(int marcador1, int marcador2) : void
#validarReglasDeporte(int marcador1, int marcador2) : void
#calcularPuntos(int marcador1, int marcador2) : int[]
+getEstado() : EstadoEvento
+getMarcador1() : Integer
+getMarcador2() : Integer
+getFecha() : LocalDate
+getHora() : int
+getSede() : String
+getEquiposParticipantes() : List<Equipo>
+getDescripcion() : String
«interface»
Puntuable
+registrarResultado(int marcador1, int marcador2) : void
Exhibicion
-String titulo
-EstadoEvento estado
-LocalDate fecha
-int hora
-String sede
+Exhibicion(String titulo)
+programar(LocalDate fecha, int hora, String sede) : void
+getEstado() : EstadoEvento
+getFecha() : LocalDate
+getHora() : int
+getSede() : String
+getDescripcion() : String
+getEquiposParticipantes() : List<Equipo>
titulo es final. Implementa los métodos de Programable salvo compareTo, que hereda. Devuelve una lista vacía de equipos. No utiliza FINALIZADO.
compareTo es default: fecha, hora y sede ascendentes. Este criterio se implementa una sola vez.
«interface»
Programable
+programar(LocalDate fecha, int hora, String sede) : void
+getFecha() : LocalDate
+getHora() : int
+getSede() : String
+getDescripcion() : String
+getEquiposParticipantes() : List<Equipo>
+compareTo(Programable otro) : int
T = Programable
«interface»
ComparableCT>
+compareTo(T otro) : int
T = Equipo
«enumeration»
EstadoEvento
SIN_PROGRAMAR
PROGRAMADO
FINALIZADO
Equipo
-String codigo
-String nombre
-String empresa
-int puntosTabla
+Equipo(String codigo, String nombre, String empresa)
+getCodigo() : String
+getNombre() : String
+getEmpresa() : String
+getPuntosTabla() : int
+sumarPuntos(int puntos) : void
+compareTo(Equipo otro) : int
compareTo implementa Comparable: puntos descendentes y nombre alfabético. codigo, nombre y empresa son final.
equipo1 y equipo2
EncuentroFutbolSala
+EncuentroFutbolSala(Equipo equipo1, Equipo equipo2)
#validarReglasDeporte(int marcador1, int marcador2) : void
#calcularPuntos(int marcador1, int marcador2) : int[]
+getDescripcion() : String
EncuentroBaloncesto
+EncuentroBaloncesto(Equipo equipo1, Equipo equipo2)
#validarReglasDeporte(int marcador1, int marcador2) : void
#calcularPuntos(int marcador1, int marcador2) : int[]
+getDescripcion() : String
Override: validarReglasDeporte, calcularPuntos y getDescripcion. Victoria: 3; empate: 1 por equipo; derrota: 0. El constructor llama a super con ambos equipos.
Override: validarReglasDeporte, calcularPuntos y getDescripcion. Rechaza empates. Victoria: 2; derrota: 1. El constructor llama a super con ambos equipos.

Prueba y estado	Datos de entrada	Resultado esperado	Resultado obtenido
P01 Datos iniciales APROBADA	Iniciar y salir con 0	4 equipos, 2 empresas, 4 encuentros (3 finalizados), 1 exhibición programada; Faro=4, Sol=2, Luna=1, Río=1	Equipos=4; empresas=2; encuentros=4; finalizados=3; exhibiciones=1; fecha exhibición=2026-11-12; tabla: Faro=4, Sol=2, Luna=1, Río=1
P02 Victoria local en fútbol sala APROBADA	Faro vs Sol, 3-1	Faro=3, Sol=0; FINALIZADO	Faro=3, Sol=0; marcador=3-1; estado=FINALIZADO
P03 Victoria visitante en fútbol sala APROBADA	Faro vs Sol, 1-3	Faro=0, Sol=3; FINALIZADO	Faro=0, Sol=3; marcador=1-3; estado=FINALIZADO
P04 Empate en fútbol sala APROBADA	Faro vs Sol, 2-2	Faro=1, Sol=1; FINALIZADO	Faro=1, Sol=1; marcador=2-2; estado=FINALIZADO
P05 Marcador cero válido APROBADA	Fútbol sala, 0-0	Faro=1, Sol=1; FINALIZADO	Faro=1, Sol=1; marcador=0-0; estado=FINALIZADO
P06 Victoria local en baloncesto APROBADA	Faro vs Sol, 78-74	Faro=2, Sol=1; FINALIZADO	Faro=2, Sol=1; marcador=78-74; estado=FINALIZADO
P07 Victoria visitante en baloncesto APROBADA	Faro vs Sol, 74-78	Faro=1, Sol=2; FINALIZADO	Faro=1, Sol=2; marcador=74-78; estado=FINALIZADO
P08 Acumulación entre deportes APROBADA	Faro gana 3-1 en fútbol; Sol gana 78-74 en baloncesto	Faro=4 y Sol=2	Faro=4, Sol=2, Luna=0, Río=0
P09 Desempate por nombre APROBADA	Faro y Luna empatan 2-2; 1 punto cada uno	Faro precede a Luna, seguido de Río y Sol con 0	Faro=1, Luna=1, Río=0, Sol=0
P10 Calendario mixto por fecha, hora y sede APROBADA	Insertar: exhibición día 20/14/C; exhibición 20/9/B; fútbol 20/9/A; baloncesto 19/20/Z finalizado; exhibición sin fecha	Baloncesto 19/20/Z; fútbol 20/9/A; exhibición 20/9/B; exhibición 20/14/C; excluir evento sin fecha	Baloncesto: Luna vs Río: fecha=2026-11-19, hora=20, sede=Z; Fútbol sala: Faro vs Sol: fecha=2026-11-20, hora=9, sede=A; Exhibición: Clínica: fecha=2026-11-20, hora=9, sede=B; Exhibición: Tarde: fecha=2026-11-20, hora=14, sede=C; eventos en calendario=4; estado baloncesto=FINALIZADO
P11 Reprogramar encuentro y liberar sede APROBADA	Fútbol día 20/9/A pasa al 21/11/B; asignar exhibición al horario anterior	Encuentro PROGRAMADO en 21/11/B, exhibición en 20/9/A; puntos intactos	Fútbol sala: Luna vs Río: fecha=2026-11-21, hora=11, sede=B, estado=PROGRAMADO, marcador=null-null; Exhibición: Clínica: fecha=2026-11-20, hora=9, sede=A; tabla antes: Faro=3, Luna=0, Río=0, Sol=0; después: Faro=3, Luna=0, Río=0, Sol=0

Prueba y estado	Datos de entrada	Resultado esperado	Resultado obtenido
P12 Reprogramar exhibición APROBADA	Clínica día 20/9/A pasa al 21/10/B	Nueva programación, PROGRAMADO; tabla intacta	Exhibición: Clínica: fecha=2026-11-21, hora=10, sede=B; estado=PROGRAMADO; tabla antes: Faro=3, Luna=0, Río=0, Sol=0; después: Faro=3, Luna=0, Río=0, Sol=0
P13 Excluir el propio evento al reprogramar APROBADA	Programar dos veces el mismo encuentro en día 20/9/A	Sin falso conflicto; misma programación y puntos	Fútbol sala: Luna vs Río: fecha=2026-11-20, hora=9, sede=A, estado=PROGRAMADO, marcador=null-null; huella antes=después: true
P14 Horas límite APROBADA	Programar exhibiciones a las 0 y 23 del mismo día y sede	Ambas aceptadas y ordenadas 00:00 antes de 23:00	Exhibición: Inicio: fecha=2026-11-20, hora=0, sede=A; Exhibición: Cierre: fecha=2026-11-20, hora=23, sede=A
P15 Conflicto de sede con exhibición APROBADA	Encuentro en día 20/9/A; intentar exhibición en 20/9/a	Rechazo por sede; tabla y eventos intactos	Rechazo: La sede ya está ocupada en esa fecha y hora. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P16 Conflicto de sede con encuentro APROBADA	Exhibición en día 20/9/A; intentar encuentro en el mismo horario y sede	Rechazo por sede; tabla y eventos intactos	Rechazo: La sede ya está ocupada en esa fecha y hora. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P17 Conflicto del primer participante entre deportes APROBADA	Fútbol Faro-Sol en día 20/9/A; baloncesto Faro-Luna en 20/9/B	Rechazo por Faro ocupado; tabla y eventos intactos	Rechazo: El equipo Faro ya tiene otro encuentro en esa fecha y hora. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P18 Conflicto del segundo participante APROBADA	Fútbol Faro-Sol en día 20/9/A; baloncesto Luna-Sol en 20/9/B	Rechazo por Sol ocupado; tabla y eventos intactos	Rechazo: El equipo Sol ya tiene otro encuentro en esa fecha y hora. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P19 Reprogramación rechazada conserva horario anterior APROBADA	Encuentro en día 20/10/B; moverlo al 20/9/A ocupado por exhibición	Rechazo; conservar 20/10/B, estado, marcador y tabla	Rechazo: La sede ya está ocupada en esa fecha y hora. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P20 Código vacío APROBADA	Registrar código de tres espacios, nombre Nuevo, empresa Sur	Rechazo; tabla y eventos intactos	Rechazo: El código del equipo no puede estar vacío. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P21 Código repetido APROBADA	Registrar código ' FAR ' ya existente	Rechazo; no duplicar ni reemplazar al equipo	Rechazo: Ya existe un equipo con el código FAR. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P22 Equipo contra sí mismo APROBADA	Crear fútbol Faro-Faro	Rechazo; tabla y eventos intactos	Rechazo: Un equipo no puede jugar contra sí mismo. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true

Prueba y estado	Datos de entrada	Resultado esperado	Resultado obtenido
P23 Participante ajeno a la copa APROBADA	Crear encuentro con otro objeto Equipo con código FAR	Rechazo; exigir la instancia registrada y conservar estado	Rechazo: Los participantes deben pertenecer a esta copa. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P24 Empate en baloncesto APROBADA	Encuentro programado; marcador 70-70	Rechazo; no guardar marcador ni repartir puntos	Rechazo: El baloncesto no permite un marcador empatado. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P25 Primer marcador negativo APROBADA	Fútbol programado, -1 a 2	Rechazo; tabla, marcador y estado intactos	Rechazo: Los marcadores deben ser enteros no negativos. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P26 Segundo marcador negativo APROBADA	Baloncesto programado, 80 a -2	Rechazo; tabla, marcador y estado intactos	Rechazo: Los marcadores deben ser enteros no negativos. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P27 Resultado sin programación APROBADA	Fútbol SIN_PROGRAMAR, marcador 2-1	Rechazo; tabla, marcador y estado intactos	Rechazo: Solo se registran resultados de encuentros programados sin resultado. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P28 Segundo resultado en finalizado APROBADA	Faro-Sol ya finalizado 3-1; intentar 0-5	Rechazo; conservar 3-1, FINALIZADO y puntos originales	Rechazo: Solo se registran resultados de encuentros programados sin resultado. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P29 Reprogramar finalizado APROBADA	Mover encuentro finalizado al día 20/9/A	Rechazo; conservar programación, resultado y tabla	Rechazo: No se puede reprogramar un encuentro finalizado. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P30 Hora negativa APROBADA	Exhibición con hora -1	Rechazo; tabla y eventos intactos	Rechazo: La hora debe estar entre 0 y 23. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P31 Hora mayor que 23 APROBADA	Encuentro con hora 24	Rechazo; tabla y eventos intactos	Rechazo: La hora debe estar entre 0 y 23. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P32 Fecha ausente APROBADA	Exhibición con fecha null	Rechazo; tabla y eventos intactos	Rechazo: La fecha es obligatoria. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true

Prueba y estado	Datos de entrada	Resultado esperado	Resultado obtenido
P33 Sede vacía APROBADA	Encuentro con sede de espacios	Rechazo; tabla y eventos intactos	Rechazo: La sede no puede estar vacía. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P34 Nombre de equipo vacío APROBADA	Código NUE, nombre vacío, empresa Sur	Rechazo; tabla y eventos intactos	Rechazo: El nombre del equipo no puede estar vacío. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P35 Empresa vacía APROBADA	Código NUE, nombre Nuevo, empresa vacía	Rechazo; tabla y eventos intactos	Rechazo: La empresa no puede estar vacía. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P36 Título de exhibición vacío APROBADA	Crear exhibición sin título	Rechazo; tabla y eventos intactos	Rechazo: El título de la exhibición no puede estar vacío. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P37 Evento no registrado APROBADA	Programar una exhibición no agregada a la copa	Rechazo; tabla y eventos intactos	Rechazo: El evento debe estar registrado en esta copa. Tabla antes: [Faro=3, Luna=0, Río=0, Sol=0]; después: [Faro=3, Luna=0, Río=0, Sol=0]. Huella antes=después: true
P38 Colecciones protegidas APROBADA	Intentar clear en tabla, eventos, encuentros y participantes; remove del iterador	5 operaciones rechazadas con UnsupportedOperationException; modelo intacto	Intentos=5; UnsupportedOperationException=5; huella antes=después: true
P39 Consola con formatos numéricos inválidos APROBADA	En el menú: abc, 1.5, entero demasiado grande, línea vacía, 6, 0	4 errores de formato; mostrar tabla y salir normalmente; datos intactos	Errores de entero=4; mensajes: Error: ingrese un número entero dentro del rango de int. / Error: ingrese un número entero dentro del rango de int. / Error: ingrese un número entero dentro del rango de int. / Error: ingrese un número entero dentro del rango de int. / Hasta pronto.; tabla observada: Faro=4, Sol=2, Luna=1, Río=1
P40 Consola con fechas inválidas APROBADA	Programar evento 4: 2026-02-30, 20/11/2026, luego 2026-11-20, hora 9, sede A	Rechazar las dos fechas, volver a pedirlas y programar sin cambiar puntos	Errores de fecha=2; mensajes: Error: ingrese una fecha válida con formato AAAA-MM-DD. / Error: ingrese una fecha válida con formato AAAA-MM-DD. / Evento programado correctamente. / Hasta pronto.; Baloncesto: Luna vs Sol: fecha=2026-11-20, hora=9, sede=A, estado=PROGRAMADO, marcador=null-null; tabla: Faro=4, Sol=2, Luna=1, Río=1

Prueba y estado	Datos de entrada	Resultado esperado	Resultado obtenido
P41 Consola rechaza horas fuera de rango APROBADA	Programar evento 4 con hora -1 y luego 24; consultar tabla y salir	Dos rechazos; evento SIN_PROGRAMAR; tabla y datos originales intactos	Rechazos de hora=2; mensajes: Error: La hora debe estar entre 0 y 23. / Error: La hora debe estar entre 0 y 23. / Hasta pronto.; estado evento 4=SIN_PROGRAMAR; tabla: Faro=4, Sol=2, Luna=1, Río=1
P42 Selecciones inválidas en el menú APROBADA	Opción 99; evento 999; resultado de encuentro -2; deporte 9; equipo XXX	Errores claros; menú continúa y no cambia datos	Mensajes: Error: opción no válida. Seleccione de 0 a 7. / Error: Evento no válido. / Error: Encuentro no válido. / Error: Deporte no válido. / Error: No existe el equipo con código XXX. / Hasta pronto.; tabla observada: Faro=4, Sol=2, Luna=1, Río=1
P43 Flujo completo del menú APROBADA	Registrar NUE/Nuevo/Sur; crear fútbol FAR-NUE y exhibición Demo; programar encuentro 5 día 20/12/A; resultado 1-0; consultar tabla y calendario; salir	5 equipos, 5 encuentros, 2 exhibiciones; encuentro nuevo FINALIZADO; Faro=7 y Nuevo=0	Equipos=5; encuentros=5; exhibiciones=2; Fútbol sala: Faro vs Nuevo: fecha=2026-11-20, hora=12, sede=A, estado=FINALIZADO, marcador=1-0; tabla: Faro=7, Sol=2, Luna=1, Río=1, Nuevo=0; mensajes: Evento programado correctamente. / Resultado registrado y puntos de tabla actualizados. / Hasta pronto.
P44 Reprogramación desde el menú APROBADA	Evento 4 al día 20/12/A y luego 21/13/B; consultar calendario	Evento 4 PROGRAMADO en 2026-11-21 13:00 B; puntos intactos	Baloncesto: Luna vs Sol: fecha=2026-11-21, hora=13, sede=B, estado=PROGRAMADO, marcador=null-null; mensajes: Evento programado correctamente. / Evento programado correctamente. / Hasta pronto.; tabla: Faro=4, Sol=2, Luna=1, Río=1
P45 Fin de entrada sin opción de salida APROBADA	Consultar tabla (6) y cerrar la entrada	Terminar con mensaje de fin de entrada; conservar datos	Mensajes: Fin de la entrada. Programa terminado.; tabla observada: Faro=4, Sol=2, Luna=1, Río=1
P46 Simultaneidad válida APROBADA	Fútbol Faro-Sol en día 20/9/A y baloncesto Luna-Río en 20/9/B	Aceptar ambos porque no comparten sede ni equipos	Fútbol sala: Faro vs Sol: fecha=2026-11-20, hora=9, sede=A; Baloncesto: Luna vs Río: fecha=2026-11-20, hora=9, sede=B; estados=PROGRAMADO,PROGRAMADO; tabla: Faro=0, Luna=0, Río=0, Sol=0
P47 Separación de interfaces APROBADA	Examinar contratos de Encuentro y Exhibicion	Ambos Programable; solo Encuentro Puntuable; exhibición sin equipos	Encuentro implementa Programable=true, Puntuable=true; Exhibicion implementa Programable=true, Puntuable=false; estado exhibición=SIN_PROGRAMAR; participantes=0